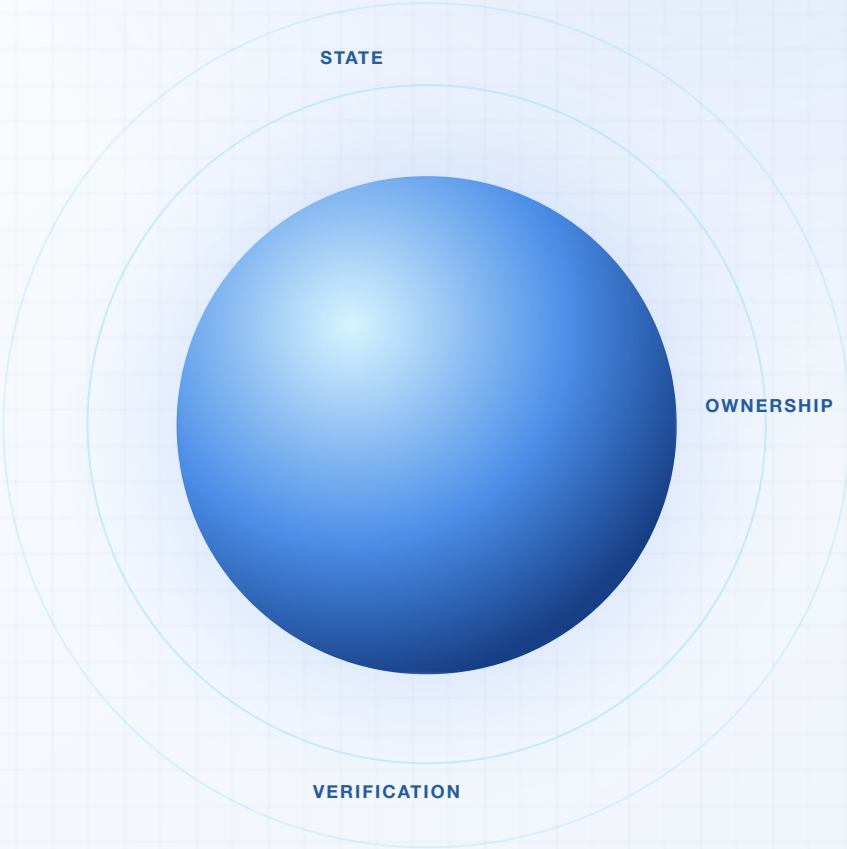
EXECUTIVE ARCHITECTURE BRIEF · TECHNICAL PREVIEW

ForgeFlow

A stateful execution protocol for AI-assisted software engineering

A protocol layer above delivery methods: governing how AI Agents make decisions, preserve engineering state and recover execution.

Agent-agnostic Core | Repository-native | Evidence-driven



Why AI Coding Needs an Engineering Protocol

Agents can draft, code, test and refactor. The remaining problem is coordinating decisions and preserving truth across long-running delivery.

WHAT AI ALREADY IMPROVED

Speed and local productivity

Rapid code generation, explanation, test drafting, refactoring and tool execution.

WHAT ENGINEERING STILL NEEDS

State, authority and recovery

Durable decisions, bounded scope, independent verification and a reliable path back upstream.

01

Context drift

Conversation is not a durable source of project truth.

02

Decision blur

Agents silently decide business, architecture and delivery trade-offs.

03

Scope expansion

One task grows into unrelated implementation without explicit authorization.

04

Weak recovery

Failures become retries instead of returning to the correct decision owner.

An Engineering Protocol for AI Agents

> Artifacts preserve state. Workflows own decisions. The Artifact Graph authorizes execution. Handoffs report validated transfers.

POSITIONING

Protocol + reference implementation

Repository-native Markdown contracts with deterministic parsing, resolution and optional orchestration.

RELATIONSHIP TO AGENTS

Agent-agnostic Core

The Core does not depend on a model API or vendor runtime. Adapters isolate host-specific invocation.

CURRENT INTEGRATION BOUNDARY

Codex and Copilot

Dedicated launchers exist today. Other agents may read the protocol, but are not claimed as validated integrations.

Leadership takeaway: ForgeFlow governs how an Agent participates in delivery; it does not replace engineering judgment, CI, source control or human accountability.

ForgeFlow does not replace the SDLC.
It governs AI participation across it.

An SDLC describes delivery stages and organizational practices. ForgeFlow defines executable rules for Agent authority, evidence, transition and recovery inside those stages.

TRADITIONAL SDLC

How software delivery is organized

Planning, requirements, design, implementation, testing, release and operations - adapted to each organization.

PROCESS MODEL

FORGEFLOW PROTOCOL

How AI execution is authorized

Who may decide, which evidence is current, what Scope may run next, and where execution must return when blocked.

EXECUTION CONTROL MODEL

> SDLC provides the organizational lifecycle. ForgeFlow provides the stateful execution protocol for AI Agents operating within it.

Positioning: ForgeFlow is protocol infrastructure above any specific SDLC methodology - not a new waterfall, Agile variant or replacement for engineering management.

Workflow / Artifact / Handoff / Ownership

The familiar Workflow, Artifact and Handoff model is completed by the Artifact Graph, which determines what is actually authorized now.

01

Workflow

Assigns exclusive responsibility for a class of engineering decisions.

WHO MAY DECIDE?

02

Artifact

Persists approved intent, requirements, design, implementation and review evidence.

WHAT IS TRUE?

03

Handoff

Reports a validated move across Workflow or Scope authorizations.

WHO TAKES OVER?

04

Artifact Graph

Validates versions, dependencies and blockers, then resolves the next authorization.

WHAT MAY RUN NOW?

A Handoff never creates truth by itself. Every participant rebuilds the graph from repository evidence and resolves authorization again.

A protocol-defined execution sequence with explicit gates

PLAN
Project intent
Feature order

›

GRILL
Observable behavior
Business clarity

›

SOLUTION
Engineering baseline
Cross-feature design

›

SLICE
Bounded delivery
Ordered slices

›

TDD
Implementation
Tests and evidence

›

REVIEW
Independent
PASS or FAIL

BUSINESS BARRIER

Planning + Requirements

Features and observable outcomes must be current before engineering design begins.

ENGINEERING BARRIER

Solution + Slicing

Architecture and delivery boundaries must be current before implementation begins.

QUALITY BARRIER

TDD + Review

Implementation evidence is independently reviewed per Slice attempt before progress continues.

This sequence is a ForgeFlow Workflow model, not a replacement for the full SDLC. Return and Recovery are protocol-level paths whenever evidence exposes an upstream gap.

Two dimensions of controlled iteration

A project is decomposed by product value and by implementation risk - without flattening both into one task list.

Feature iteration

Planning declares stable Feature IDs and delivery order.
Each Feature receives its own Requirement contract.

Slice iteration

Each Feature receives an ordered Slice Plan.
Every Slice advances through TDD and independent Review.

Deterministic completion: Planning defines Feature order. Slice Plans define Slice order. A single successful Slice can never be mistaken for project completion.

Every decision has one authoritative owner

Ownership prevents the common AI failure mode of making every decision locally because it can.

PLAN	Project intent, Feature order, exclusions and incremental Change Set	PROJECT
GRILL	Business behavior, acceptance criteria and unresolved ambiguity	FEATURE
SOLUTION	Architecture, interfaces, data, security, operations and verification strategy	PROJECT
SLICE	Bounded implementation slices and their declared execution order	FEATURE
TDD	Implementation and tests for one authorized Slice attempt	SLICE
REVIEW	Independent PASS/FAIL decision and routed findings	SLICE ATTEMPT

Failures return to the owner of the missing decision

RETURN ROUTING

Business blocker	Return to Grill - or Plan for project scope
Engineering blocker	Return to Solution
Slice boundary	Return to Slice
Implementation issue	Return to TDD

RECOVERY WITHOUT CONVERSATION MEMORY

Restart from evidence

A new process rebuilds the Artifact Graph, validates the current persisted blocker and re-authorizes only the correct owner and scope.

Runtime history is diagnostic. Repository evidence remains the recovery authority.

> ForgeFlow distinguishes retrying execution from correcting the decision that made execution impossible.

Engineering knowledge becomes a versioned evidence chain

Planning

Intent, Feature order, constraints, Change Set

Requirement

Behavior, acceptance criteria, business contract version

Solution Plan

Engineering baseline and per-Feature engineering versions

Slice Plan

Ordered delivery boundaries for one Feature

TDD Record

Implementation attempt, tests and current upstream versions

Review Report

Immutable attempt, findings, PASS/FAIL and Return disposition

Current truth

Only canonical paths and valid current versions participate in authorization.

Historical trace

Replaced contracts and supplemental reports move to dedicated history roots.

Scoped change

Feature contract and engineering versions preserve unaffected downstream evidence.

Governance is embedded in execution, not added after it

Scope	Typed Project, Feature and Slice authorization prevents silent expansion.
Decision	Exclusive Artifact ownership prevents agents from approving their own assumptions.
Quality	TDD evidence and independent Review create a per-attempt quality gate.
Traceability	Version edges connect intent, requirement, design, implementation and verification.
AI overreach	Invalid, stale or contradictory evidence fails closed instead of authorizing work.
Extensibility	Markdown Review plugins add ordered checks without transferring PASS/FAIL ownership.

Important boundary: ForgeFlow reduces workflow risk. It does not guarantee model correctness or replace normal security, branch protection, CI and human review.

Copilot / Claude / Codex independent by design

ForgeFlow's protocol truth lives in repository contracts, not in a model, prompt product or vendor runtime.

1

Stable Core

Workflow, Artifact, Handoff and Ownership semantics remain host-independent.

2

Thin Adapters

Launchers translate host invocation into the same protocol-defined Workflow entry.

3

Replaceable Agents

Teams can change Agents without replacing engineering state or governance contracts.

Validated today

Dedicated ForgeFlow launchers and adapter content exist for Codex and GitHub Copilot.

Protocol-compatible direction

Claude and other Agents can participate through conforming adapters, but are not presented here as currently validated integrations.

Agent independence protects the durable investment: engineering evidence, decision ownership and transition rules remain portable.

From AI coding assistance to governed AI engineering

The strategic direction is a portable execution protocol that makes AI participation observable, replaceable and accountable across the software lifecycle.

STRENGTHEN THE CORE

Stable protocol and test matrix

Freeze versioned contracts, expand transition coverage and preserve deterministic behavior across implementations.

DIRECTION - NOT A CURRENT CLAIM

BROADEN THE EDGE

More validated Adapters

Add host integrations only after verifying real discovery, invocation, permissions and acceptance behavior.

DIRECTION - NOT A CURRENT CLAIM

OPERATIONALIZE GOVERNANCE

Safer automation and upgrade paths

Improve distribution, refresh, evidence inspection and controlled automation without creating a second source of truth.

DIRECTION - NOT A CURRENT CLAIM

> From AI Coding to Protocol-Driven AI Engineering: the long-term asset is not one smarter Agent, but a system that safely coordinates changing Agents.